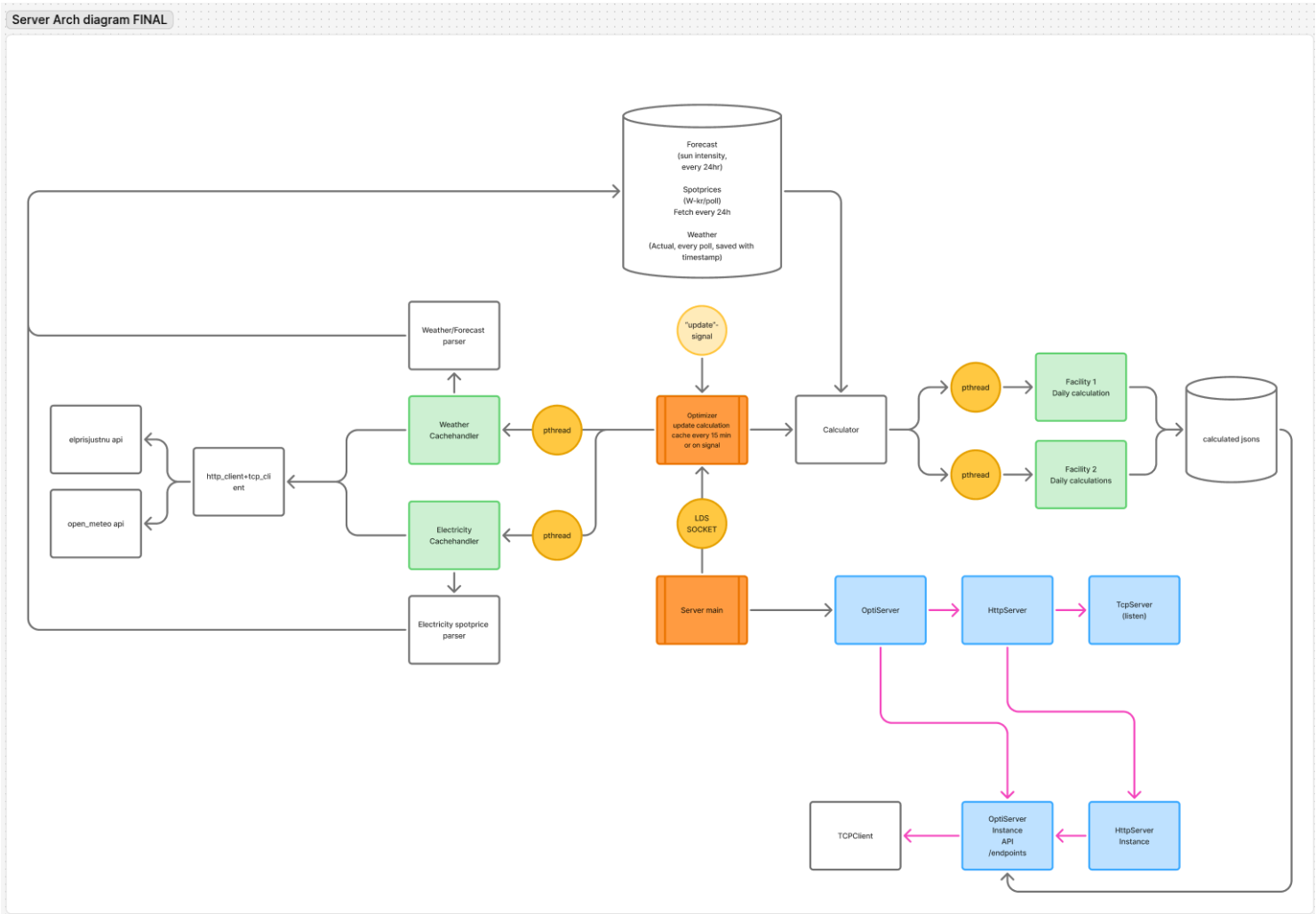
# OptiMaestro Teknisk Dokumentation

**Länk till Figma**

**Repon:**

- **MaestroCore (Allmäna verktyg och moduler)**
- **OptiMaestro (Huvudprojekt)**
- **GUIMaestro (C++ Klient)**

# Arkitektur




Vi valde att köra detta system i 3 separata processer. Optimizer som hanterar hämtning av data, cachning samt algoritmer för energianvändning. Server som tillhandahåller API-endpoints för att hämta data, uppdatera konfiguration eller skapa nya anläggningar. En klient som kan skapa nya anläggningar samt hämta energirapport för nuvarande.

**Optimizer**

Optimizern laddar konfig för alla anläggningar, hämtar ny data, cachar den, kör algoritmer och genererar energirapporter. Detta sker vid uppstart, på request samt var 15:e minut.

Trådarna skapas i en pool, som körs allt eftersom. Det skapas en intern kö, trådarna som nämns nedan hamnar i den poolen.

Till skillnad från servern kör optimizern blocking I/O då vi hanterar simultana operationer med trådar. Det skapas 4 trådar som hämtar spotprisdata för alla energiklasser(prisklasser) samt 2 st trådar för varje anläggning som hämtar väderdata (forecast + current). Hämtning sker via vår egen http-klient som använder tls-lager implementerat med mbedtls. Datan cachas i en sqlite3 databas, den innehåller 3 tabeller: electricity_spots, facility & weather_values;

electricity_spots = spotpriser för alla prisklasser

facility = anläggningsspecifika värden, som lat/lon

weather_values = temperatur, vind, sol samt facility_id som knyter väderdatan till rätt anläggning

När data har hämtats och cachats stänger vi ner trådarna och nya trådar skapas för att köra beräkningar.

Det skapas 2 trådar per anläggning, dessa hämtar ut data ur sqlite databasen för sin anläggning, kör sedan beräkningarna och skapar energirapporter som lagras i en delad folder.

**Server**

Servern drivs av en statemachine (scheduler) och varje modul skapar tasks som läggs i kö och får jobba lite grann varje varv.

Den består av 3 delar, ett TCP-lager som skapar socket och väntar på inkommande anslutningar, ett HTTP lager som tolkar request och skickar vidare till rätt endpoint samt ett “opti” lager som tillhandahåller api endpoints för klienter för att hämta/läsa rapporter samt skapa nya eller uppdater anläggningar.

Om en ny anläggning skapas skickar servern via en unix domain socket ett meddelande till optimizern att hämta ny väderdata för den nya anläggningen, samt genomföra beräkningar och generera en energirapport.

När en klient request kommer in kollar servern ifall det finns en energirapport för den anläggningen och returnerar det i bodyn.

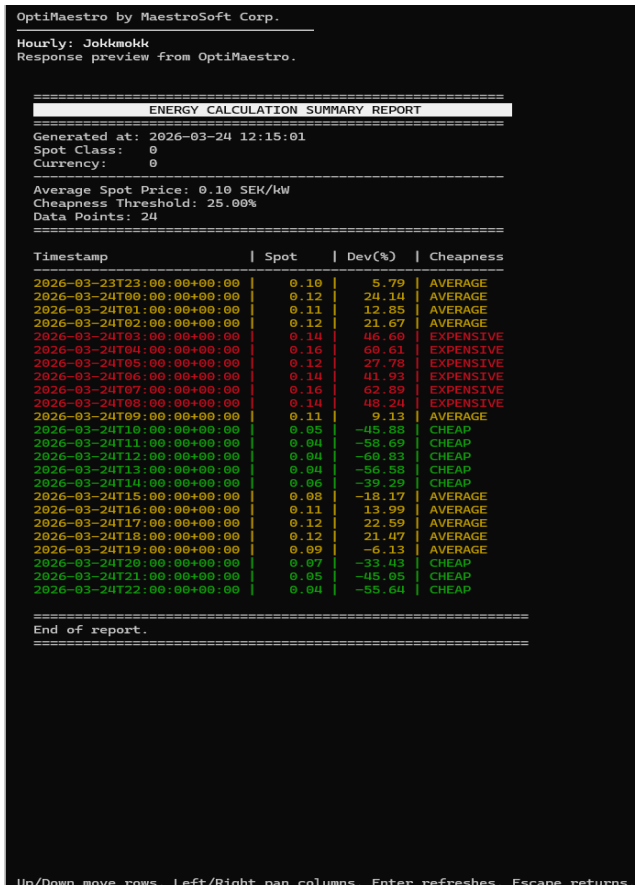
Även servern använder vår egen http-klient med mbedtls lager, dock kör servern en non-blocking version eftersom den drivs av en statemachine på en tråd, så varje task får göra en sak sedan går vi vidare till nästa, vi har ingen blockerande IO.

**Klient**

Klienten är terminalbaserad och skriven i C++ och är tänkt att köras parallellt med OptiMaestro. Syftet är att ge användaren ett enkelt UI där man kan lista anläggningar, skapa nya eller ändra befintliga anläggningskonfigurationer, samt hämta beräkningsrapporter från OptiMaestro-servern utan att behöva arbeta direkt i konfigurationsfiler eller via råa HTTP-anrop.

Användargränssnittet är byggt som en fristående klient i C++ med ncurses för att skapa ett interaktivt UI. Det är uppdelat i en klientdel (client_cpp/) där skärmflöden, tangenthantering, konfigurationshantering och API-anrop görs, samt en verktygsdel (utils/) som kapslar in ncurses. Klienten kommunicerar med OptiMaestro över TCP och skickar enkla HTTP-förfrågningar till serverns API för att till exempel läsa in anläggningskonfiguration som uppdaterats eller skapats, samt för att hämta kalkyleringsrapporter.

Funktionellt är klienten anpassningsbar för att köras både lokalt eller externt. Den har stöd för både horisontell och vertikal scrollning samt en färgkodad representation av rapporterna.



```
OptiMaestro by MaestroSoft Corp.

Hourly: Jokkmokk
Response preview from OptiMaestro.

  ==========================================================
                 ENERGY CALCULATION SUMMARY REPORT
  ==========================================================
  Generated at: 2026-03-24 12:15:01
  Spot Class:   0
  Currency:     0
  ----------------------------------------------------------
  Average Spot Price: 0.10 SEK/kW
  Cheapness Threshold: 25.00%
  Data Points: 24
  ==========================================================

  Timestamp                  | Spot     | Dev(%)   | Cheapness
  ----------------------------------------------------------
  2026-03-23T23:00:00+00:00 |     0.10 |     5.79 | AVERAGE
  2026-03-24T00:00:00+00:00 |     0.12 |    24.14 | AVERAGE
  2026-03-24T01:00:00+00:00 |     0.11 |    12.85 | AVERAGE
  2026-03-24T02:00:00+00:00 |     0.12 |    21.67 | AVERAGE
  2026-03-24T03:00:00+00:00 |     0.14 |    46.60 | EXPENSIVE
  2026-03-24T04:00:00+00:00 |     0.16 |    60.61 | EXPENSIVE
  2026-03-24T05:00:00+00:00 |     0.12 |    27.78 | EXPENSIVE
  2026-03-24T06:00:00+00:00 |     0.14 |    41.93 | EXPENSIVE
  2026-03-24T07:00:00+00:00 |     0.16 |    62.89 | EXPENSIVE
  2026-03-24T08:00:00+00:00 |     0.14 |    48.24 | EXPENSIVE
  2026-03-24T09:00:00+00:00 |     0.11 |     9.13 | AVERAGE
  2026-03-24T10:00:00+00:00 |     0.05 |   -45.88 | CHEAP
  2026-03-24T11:00:00+00:00 |     0.04 |   -58.69 | CHEAP
  2026-03-24T12:00:00+00:00 |     0.04 |   -60.83 | CHEAP
  2026-03-24T13:00:00+00:00 |     0.04 |   -56.58 | CHEAP
  2026-03-24T14:00:00+00:00 |     0.06 |   -39.29 | CHEAP
  2026-03-24T15:00:00+00:00 |     0.08 |   -18.17 | AVERAGE
  2026-03-24T16:00:00+00:00 |     0.11 |    13.99 | AVERAGE
  2026-03-24T17:00:00+00:00 |     0.12 |    22.59 | AVERAGE
  2026-03-24T18:00:00+00:00 |     0.12 |    21.47 | AVERAGE
  2026-03-24T19:00:00+00:00 |     0.09 |    -6.13 | AVERAGE
  2026-03-24T20:00:00+00:00 |     0.07 |   -33.43 | CHEAP
  2026-03-24T21:00:00+00:00 |     0.05 |   -45.05 | CHEAP
  2026-03-24T22:00:00+00:00 |     0.04 |   -55.64 | CHEAP

  ==========================================================
  End of report.
  ==========================================================

Up/Down move rows  Left/Right pan columns  Enter refreshes  Escape returns
```

# Profilering

Det vi valde att profilera främst var optimizern då den var viktigast att gå snabbt för att sömlöst hämta nya beräkningar. I och med att den vanligtvis körs daemoniserad och tar tid i en oändlig loop så bestämde vi oss för att skapa en alternativ main() som körde linjärt genom hela hämtningsprocessen.

Resultatet visade att det som tog längst tid var externa funktioner som vi inte har någon avsikt att ändra i, nämligen sqlite3 samt mbedTLS, och det var svårt att urskilja vad som tog lång tid av våra egna funktioner.

Även i de externa funktionerna var self seconds så pass låg att  vi inte såg någon anledning att minska användandet för att optimera på det sättet.

**gprof resultat:**

```
Flat profile:

Each sample counts as 0.01 seconds.
  %   cumulative   self              self     total
 time   seconds   seconds    calls  ms/call  ms/call  name
 28.57      0.02     0.02     2364     0.01     0.01  sqlite3VdbeExec
 14.29      0.03     0.01     2147     0.00     0.00  sqlite3VdbeMemSetInt64
 14.29      0.04     0.01     1338     0.01     0.01  keywordCode
 14.29      0.05     0.01                             mbedtls_base64_decode
 14.29      0.06     0.01                             mbedtls_ct_base64_dec_value
 14.29      0.07     0.01                             mbedtls_ct_uchar_in_range_if
  0.00      0.07     0.00    21595     0.00     0.00  pthreadMutexEnter
  0.00      0.07     0.00    21595     0.00     0.00  pthreadMutexLeave
  0.00      0.07     0.00    21595     0.00     0.00  sqlite3_mutex_enter
  0.00      0.07     0.00    21595     0.00     0.00  sqlite3_mutex_leave
  0.00      0.07     0.00    15793     0.00     0.00  sqlite3VdbeOneByteSerialTypeLen
  0.00      0.07     0.00    14084     0.00     0.00  sqlite3VdbeMemRelease
  0.00      0.07     0.00    12774     0.00     0.00  sqlite3VdbeRealValue
  0.00      0.07     0.00    12536     0.00     0.00  sqlite3VdbeSerialGet
  0.00      0.07     0.00    11665     0.00     0.00  vdbeRecordDecodeInt
  0.00      0.07     0.00    11184     0.00     0.00  sqlite3ApiExit
  0.00      0.07     0.00    11176     0.00     0.00  sqlite3BtreeCursorHasMoved
  0.00      0.07     0.00    10542     0.00     0.00  applyAffinity
  0.00      0.07     0.00    10154     0.00     0.00  getCellInfo
  0.00      0.07     0.00     9929     0.00     0.00  sqlite3VdbeSerialTypeLen
  0.00      0.07     0.00     9886     0.00     0.00  columnMallocFailure
  0.00      0.07     0.00     9886     0.00     0.00  columnMem
  0.00      0.07     0.00     9721     0.00     0.00  vdbeRecordCompareInt
  0.00      0.07     0.00     9520     0.00     0.00  vdbeSafety
  0.00      0.07     0.00     9482     0.00     0.00  vdbeSafetyNotNull
  0.00      0.07     0.00     8682     0.00     0.00  sqlite3RealToI64
  0.00      0.07     0.00     8682     0.00     0.00  sqlite3VdbeIntegerAffinity
  0.00      0.07     0.00     8667     0.00     0.00  sqlite3VdbeRecordCompareWithSkip
  0.00      0.07     0.00     7565     0.00     0.00  fetchPayload
  0.00      0.07     0.00     7565     0.00     0.00  sqlite3BtreePayloadFetch
  0.00      0.07     0.00     7565     0.00     0.00  sqlite3BtreePayloadSize
  0.00      0.07     0.00     7118     0.00     0.00  vdbeUnbind
gprof.txt
```

```
Flat profile:

Each sample counts as 0.01 seconds.
  %   cumulative   self              self     total
 time   seconds   seconds    calls  Ts/call  Ts/call  name
 20.00      0.01     0.01                             ecp_mod_p256
 20.00      0.02     0.01                             mbedtls_ct_uchar_in_range_if
 20.00      0.03     0.01                             mbedtls_mpi_core_mla
 20.00      0.04     0.01                             mbedtls_mpi_core_shift_r
 20.00      0.05     0.01                             mbedtls_mpi_core_sub
  0.00      0.05     0.00    34115     0.00     0.00  pthreadMutexEnter
  0.00      0.05     0.00    34115     0.00     0.00  pthreadMutexLeave
  0.00      0.05     0.00    34115     0.00     0.00  sqlite3_mutex_enter
  0.00      0.05     0.00    34115     0.00     0.00  sqlite3_mutex_leave
  0.00      0.05     0.00    31859     0.00     0.00  sqlite3VdbeMemRelease
  0.00      0.05     0.00    25281     0.00     0.00  applyAffinity
  0.00      0.05     0.00    25268     0.00     0.00  vdbeRecordCompareInt
  0.00      0.05     0.00    21567     0.00     0.00  vdbeRecordDecodeInt
  0.00      0.05     0.00    19837     0.00     0.00  sqlite3VdbeSerialTypeLen
  0.00      0.05     0.00    19656     0.00     0.00  sqlite3RealToI64
  0.00      0.05     0.00    19656     0.00     0.00  sqlite3VdbeIntegerAffinity
  0.00      0.05     0.00    19433     0.00     0.00  vdbeSafety
  0.00      0.05     0.00    19377     0.00     0.00  vdbeSafetyNotNull
  0.00      0.05     0.00    16964     0.00     0.00  sqlite3VdbeOneByteSerialTypeL
en
  0.00      0.05     0.00    16524     0.00     0.00  sqlite3VdbeRealValue
  0.00      0.05     0.00    16155     0.00     0.00  sqlite3VdbeRecordCompareWithS
kip
  0.00      0.05     0.00    16031     0.00     0.00  sqlite3VdbeMemTooBig
  0.00      0.05     0.00    16027     0.00     0.00  vdbeUnbind
  0.00      0.05     0.00    14154     0.00     0.00  sqlite3VdbeSerialGet
  0.00      0.05     0.00    13873     0.00     0.00  sqlite3DbMallocRawNN
  0.00      0.05     0.00    13556     0.00     0.00  sqlite3VdbeMemShallowCopy
  0.00      0.05     0.00    13293     0.00     0.00  sqlite3DbNNFreeNN
  0.00      0.05     0.00    13287     0.00     0.00  sqlite3BtreeCursorHasMoved
  0.00      0.05     0.00    13199     0.00     0.00  sqlite3ApiExit
  0.00      0.05     0.00    12939     0.00     0.00  sqlite3VdbeMemSetNull
  0.00      0.05     0.00    12669     0.00     0.00  getCellInfo
  0.00      0.05     0.00    12325     0.00     0.00  sqlite3IsNaN
  0.00      0.05     0.00    12147     0.00     0.00  sqlite3VdbeMemSetDouble
  0.00      0.05     0.00    11987     0.00     0.00  sqlite3_bind_double
  0.00      0.05     0.00    10939     0.00     0.00  sqlite3_free
  0.00      0.05     0.00     9943     0.00     0.00  sqlite3GetVarint
  0.00      0.05     0.00     9931     0.00     0.00  sqlite3DbMallocRaw
  0.00      0.05     0.00     9890     0.00     0.00  columnMallocFailure
  0.00      0.05     0.00     9890     0.00     0.00  columnMem
:
```

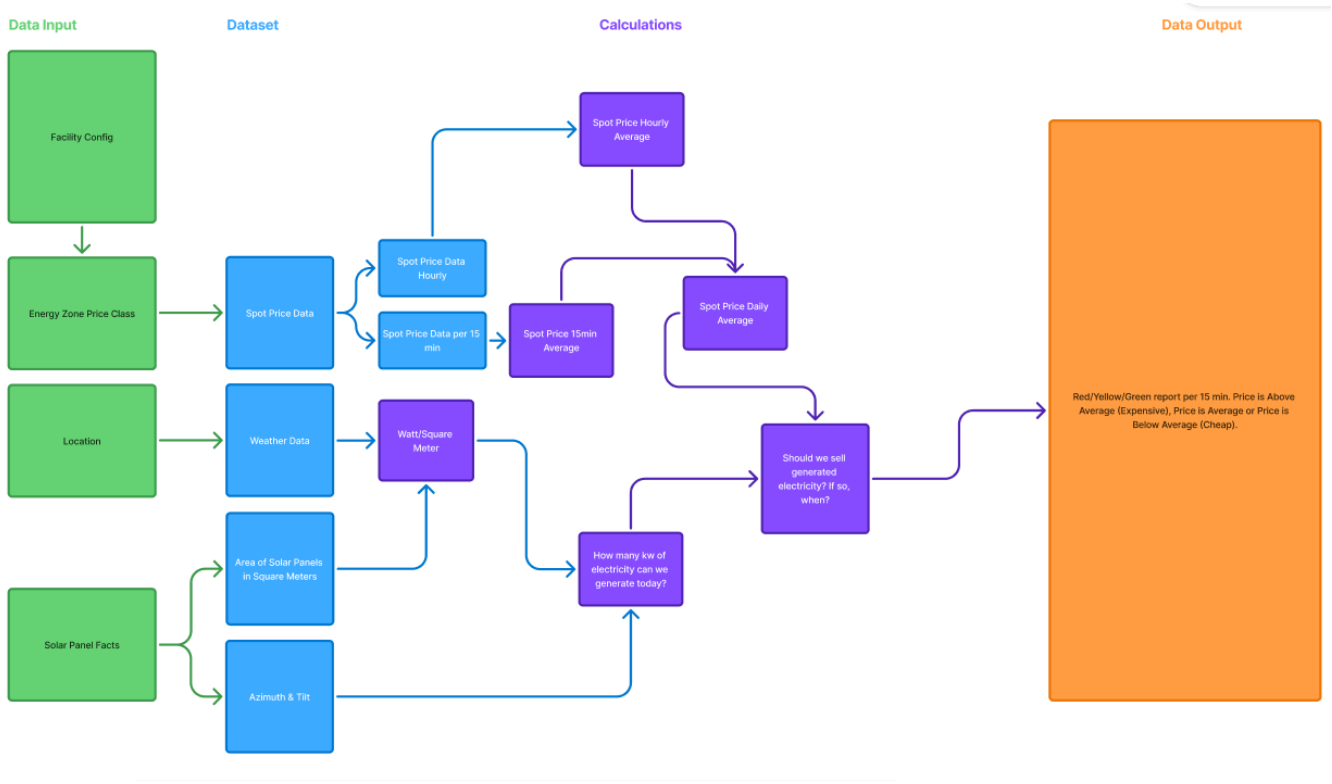
**perf resultat:**

Vi körde även samma linjära process med perf för att se hur många cache missar vi får kontra cpu cykler och instruktioner och får ett acceptabelt resultat:

```
Performance counter stats for 'optimizer':

              42.35 msec task-clock:u

       149,340,088      cycles:u

       413,255,014      instructions:u

           435,770      cache-misses:u

           305,956      branch-misses:u


       5.144196727 seconds time elapsed

       0.034143000 seconds user
       0.008839000 seconds sys
```

Av resultaten valde vi att inte göra några särskilda optimeringar då det inte kändes värt mödan, eftersom systemet redan presterade så bra som vi hade hoppats. Kanske ifall det blir relevant att skala till större mängd data, antingen i form av hundra-/tusentals fler anläggningar och/eller längre tidsperioder än dygnsberäkningar, så kan det vara värt att återbesöka profileringen.

# Beräkningar




Beräkningarna som görs baseras på två huvudsakliga indata-källor:

**Elspot-priser**

Det främsta vi vill veta är hur dyr elförbrukning är en given kvart/timme i förhållande till dagens medelvärde. Detta ges av tjänsten “elprisjustnu” och presenteras i formatet SEK/kW.

**Solstrålning**

Denna är beroende av spotpriserna, eftersom utdatan visar på hur mycket pengar vi tjänar på storleken och placeringen av våra solpaneler med hur dyr elen är den kvarten/timmen. Denna data representeras i form av SEK tjänade per given kvart/timme.

# Minneshantering

Vi har flitigt använt oss av valgrind för att snappa upp minnesläckage och -errors genom varje iteration.

Nedan från en testkörning med olika hämtningar och reloads.

```
==275648== LEAK SUMMARY:
==275648==    definitely lost: 0 bytes in 0 blocks
==275648==    indirectly lost: 0 bytes in 0 blocks
==275648==      possibly lost: 0 bytes in 0 blocks
==275648==    still reachable: 7,760 bytes in 17 blocks
==275648==         suppressed: 0 bytes in 0 blocks
==275648==
==275648== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Det som indikeras vara fortsatt nåbart minne här är från det externa biblioteket mbedTLS som likt curl inte hinner rensa allt innan programmet avslutats, trots gedigen mitigering. Men det viktigaste är att allt dynamiskt minne vi allokerat har frigjorts, att inga oinitierade variabler skrivits till eller buffrar överskridits etc.